CE QUE J'AI COMPRIS DE LA SPÉCIFICATION

Vehicle Connectivity SDK

Position, outils, obligations, langage, diagrammes et nouvelles notions

---

Rôle : Développeur 1 — Vehicle Connectivity SDK

Réalisé par : Aymen

École : ENI Carthage — Infotronique, Systèmes Embarqués

Entreprise d'accueil : Amsys Consulting, El Fahs

Encadrant : Ayoub Mejri

Langage retenu : C++

Ce document synthétise ma compréhension de la spécification fonctionnelle et technique du Vehicle Connectivity SDK, en vue de la présenter à l'encadrant avant le démarrage de l'implémentation.

Sommaire

Chapitre 1 — Position de ma tâche dans l'architecture 3

Chapitre 2 — Ce qui est obligatoire à faire 4

Chapitre 3 — Choix du langage : C++ 4

Chapitre 4 — Outils et bibliothèques nécessaires 5

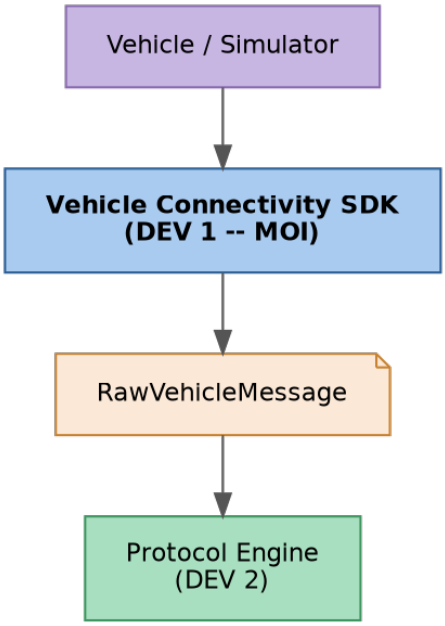
Chapitre 5 — Nouvelles notions à maîtriser 5

Chapitre 6 — Diagrammes 6

Chapitre 7 — Nouveaux fichiers et code à ajouter 7

Chapitre 1 — Position de ma tâche dans l'architecture

Le SDK que je dois développer est une couche d'abstraction, strictement positionnée entre le véhicule (ou son simulateur) et le moteur de protocoles du Développeur 2. Je ne dois jamais interpréter le contenu d'une commande ni d'une réponse — seulement les transporter de façon fiable.



Le SDK s'arrête exactement à RawVehicleMessage ; le décodage (ex: 410C1AF8 vers 1726 RPM) appartient au Développeur 2.

Le SDK doit rester entièrement découplé du moteur de diagnostic, du moteur IA, du stockage, du frontend et du modèle prédictif.

Chapitre 2 — Ce qui est obligatoire à faire

D'après la spécification, le développement est considéré terminé lorsque tous les critères suivants sont validés :

#	Critère obligatoire
1	Au moins deux types de connecteurs fonctionnent
2	Le Virtual Connector fonctionne avec un simulateur
3	La découverte des interfaces (discovery) fonctionne
4	Une VehicleSession peut être créée
5	Les commandes peuvent être envoyées via send()
6	Les réponses sont reçues sans décodage métier
7	Les timeouts sont gérés (connexion, commande, lecture, reconnexion)
8	La perte de connexion est détectée
9	La reconnexion automatique fonctionne (retry + backoff)
10	Les erreurs sont normalisées (catalogue ConnectorError)
11	Les métriques techniques sont disponibles
12	Les logs sont structurés (JSON)
13	Le mode READ_ONLY est actif par défaut
14	Les tests automatisés passent
15	Un nouveau connecteur peut être ajouté sans modifier le Core

Chapitre 3 — Choix du langage : C++

Le langage retenu pour l'implémentation du SDK est **C++**.

Critère	Pourquoi C++ convient
Performance	Traitement bas niveau (sockets, port série, CAN) sans surcoût d'interpréteur
Portabilité vers cible embarquée	Exécution native possible sur carte embarquée sans runtime additionnel
Contrôle mémoire	Maîtrise fine du cycle de vie des connexions et des buffers (SDK longue durée)
Polymorphisme natif	Les patterns Strategy / Factory / Adapter / State s'expriment via classes abstraites et héritage
Écosystème embarqué automobile	Langage standard dans l'industrie automobile et les systèmes embarqués temps réel

Chapitre 4 — Outils et bibliothèques nécessaires

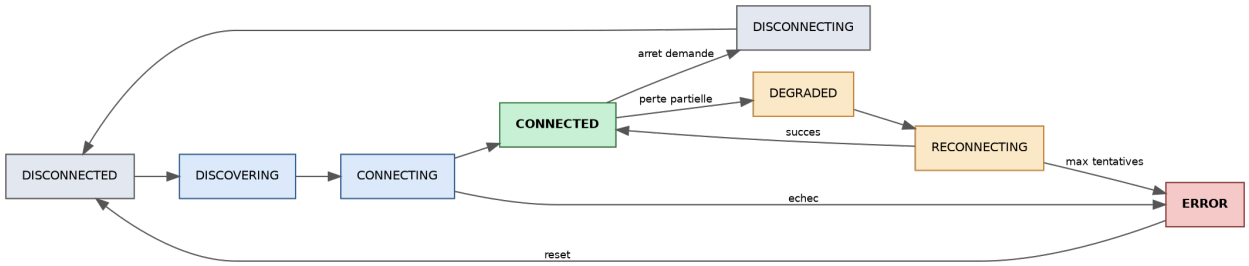
Besoin	Outil / bibliothèque C++
Compilation	CMake (système de build multiplateforme)
Sockets TCP/IP	Boost.Asio ou sockets POSIX natifs
Port série (Serial Connector)	libserialport ou Boost.Asio (serial_port)
Bluetooth Classic / BLE	BlueZ (D-Bus, Linux) pour Classic ; un stack BLE (ex. SimpleBLE) pour BLE
SocketCAN	API native Linux (linux/can.h, linux/can/raw.h)
Sérialisation JSON	nlohmann/json
Configuration YAML	yaml-cpp
Logging structuré	spdlog (support JSON, niveaux DEBUG à CRITICAL)
Tests unitaires et contractuels	Google Test (gtest)
Tests de performance	Google Benchmark
Gestion de versions	Git / dépôt partagé de l'équipe

Chapitre 5 — Nouvelles notions à maîtriser

Notion	Ce que c'est
RawVehicleMessage	Objet standardisé représentant une réponse brute (messageId, sessionId, rawPayload, status, latency...)
VehicleSession	Encapsule le cycle de vie complet d'une connexion (sessionId, statistics, errorCount...)
ConnectorFactory	Pattern Factory : instancie le bon connecteur selon la configuration, sans exposer sa classe concrète
CommandSecurityPolicy	Valide chaque commande avant émission ; bloque par défaut les commandes modificatrices (mode READ_ONLY)
Discovery	Recherche automatique des interfaces disponibles (Bluetooth, ports série, interfaces CAN, réseau)
Retry avec exponential backoff + jitter	Politique de reconnexion progressive (délai qui augmente, avec variation aléatoire)
Rate limiting	Contrôle du débit de commandes (ex. max 20 requêtes/s) pour ne pas saturer le véhicule
Event Bus / Observer	Publication d'évènements (ConnectionEstablished, MessageReceived...) consommés par d'autres composants
Machine d'état à 8 états	DISCONNECTED, DISCOVERING, CONNECTING, CONNECTED, DEGRADED, RECONNECTING, ERROR, DISCONNECTING

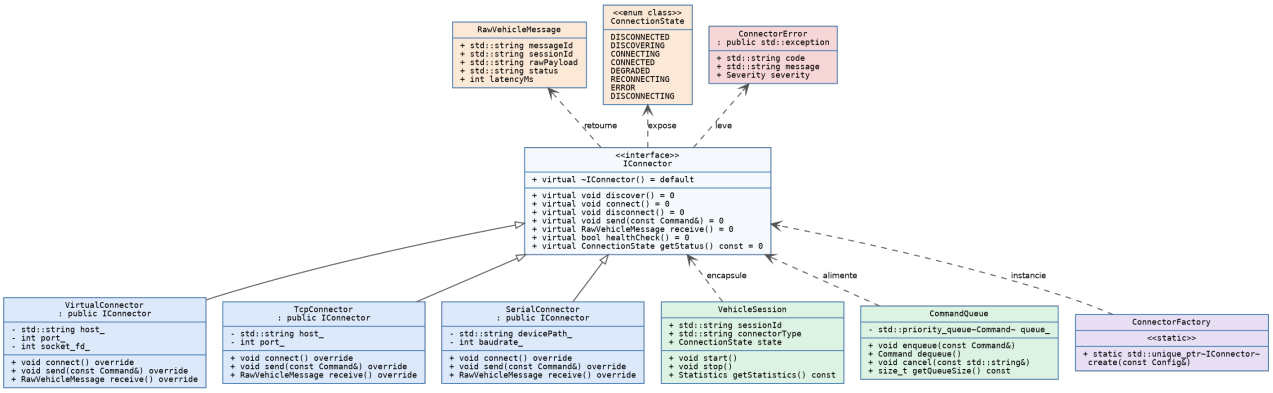
Chapitre 6 — Diagrammes

6.1 Machine d'état de connexion



Les 8 états imposés par la spécification et leurs transitions.

6.2 Diagramme de classes (C++)



IConnector comme interface pure (méthodes virtuelles = 0), avec ses implémentations et objets associés.

Chapitre 7 — Nouveaux fichiers et code à ajouter

Structure de projet recommandée par la spécification, adaptée aux conventions C++ (CMake, séparation include/src) :

```
vehicle-connectivity-sdk/
|-- CMakeLists.txt
|-- include/vcsdk/
|   |-- core/
|   |   |-- IConnector.hpp
|   |   |-- VehicleSession.hpp
|   |   |-- RawVehicleMessage.hpp
|   |   |-- Command.hpp
|   |   `-- ConnectorEvent.hpp
|   |-- connectors/
|   |   |-- VirtualConnector.hpp
|   |   |-- TcpConnector.hpp
|   |   |-- SerialConnector.hpp
|   |   |-- BluetoothConnector.hpp
|   |   `-- SocketCanConnector.hpp
|   |-- discovery/InterfaceDiscovery.hpp
|   |-- queue/CommandQueue.hpp
|   |-- retry/RetryPolicy.hpp
|   |-- buffer/LocalBuffer.hpp
|   |-- security/CommandSecurityPolicy.hpp
|   |-- monitoring/Metrics.hpp
|   `-- logging/StructuredLogger.hpp
|-- src/          (implementations .cpp correspondantes)
|-- tests/
|   |-- unit/
|   |-- contract/
|   |-- integration/
|   `-- performance/
`-- examples/
    `-- demo_virtual_connector.cpp
```

Première brique à coder : l'interface `IConnector` et le `VirtualConnector` — rien d'autre ne peut être testé sans eux.